UNIVERSIDAD CATÓLICA ARGENTINA
SANTA MARÍA DE LOS BUENOS AIRES

Facultad de Ingeniería y Ciencias Agrarias

Carrera: Ingeniería Informática

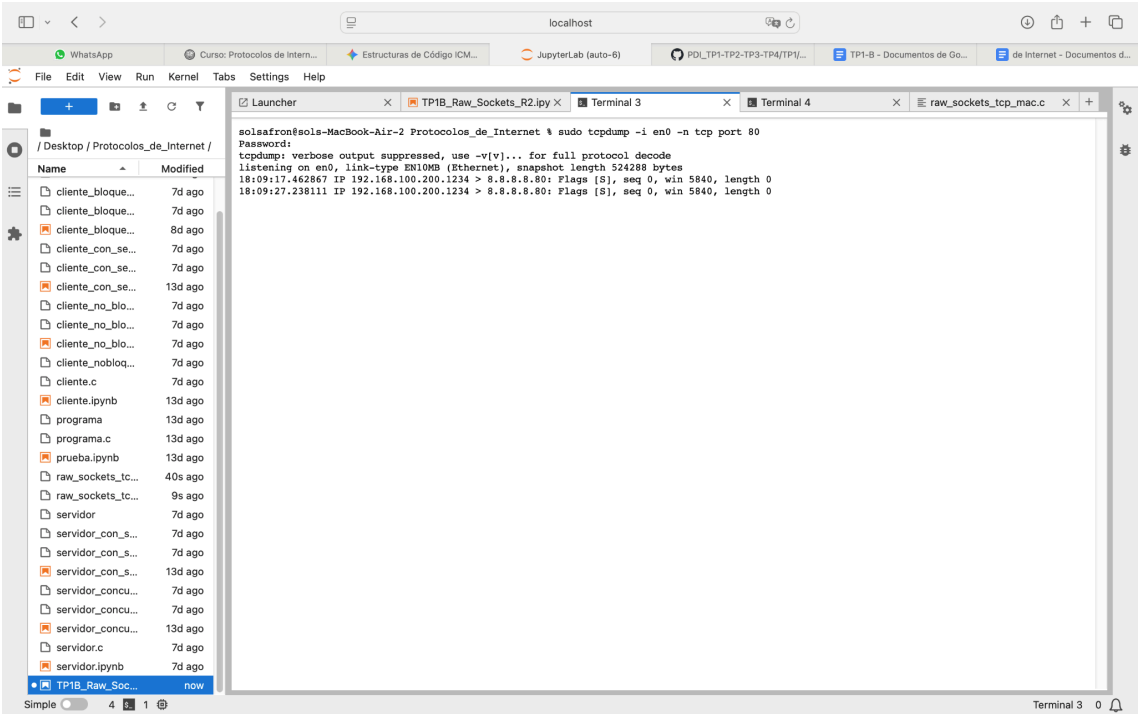
Cátedra: Protocolos de Internet

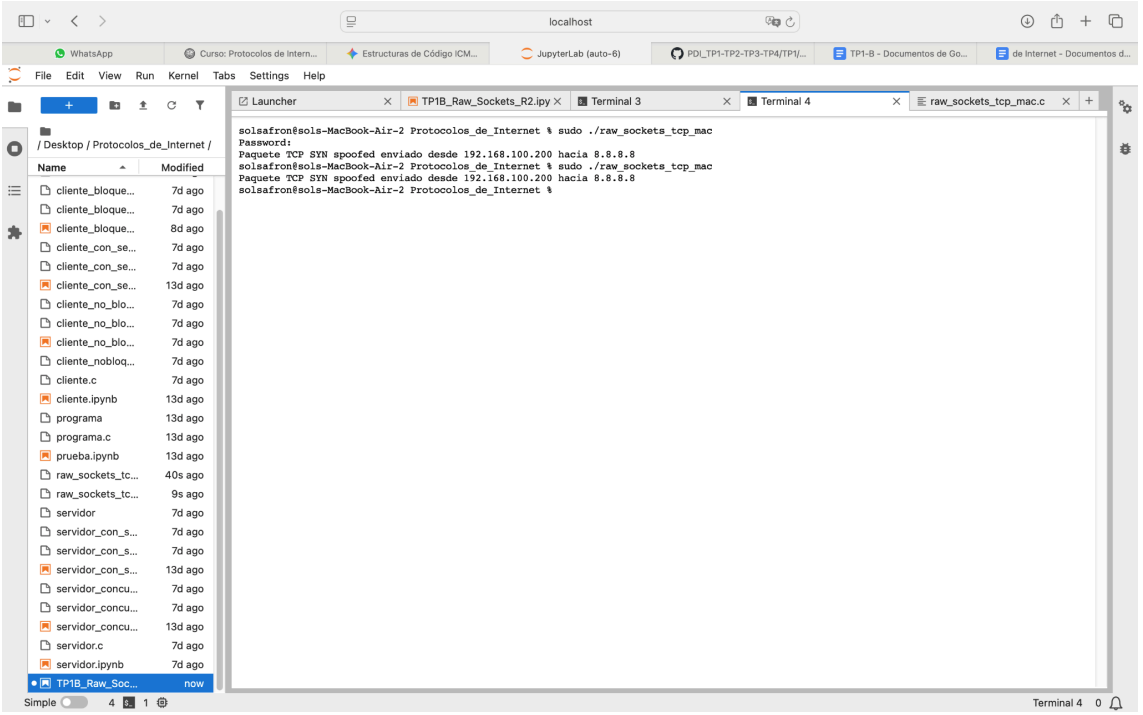
PRÁCTICA 1 – Cliente Servidor

Fecha: 13 / 04 / 2026

Estudiante/s: Guillermo Alberto Rizzuto, Agustín Martín Safronchik, Gonzalo Val

Legajo/s: 45129620, 44517212, 151722804






```
%%bash
cat << 'EOF' > raw_sockets_tcp_mac.c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
```

```
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/tcp.h>

// Calcula checksum para IP/TCP
unsigned short checksum(unsigned short *ptr, int nbytes) {
   long sum = 0;
   unsigned short oddbyte;
   unsigned short answer;

   while(nbytes > 1) {
      sum += *ptr++;
      nbytes -= 2;
   }

   if(nbytes == 1) {
      oddbyte = 0;
      *((unsigned char*)&oddbyte) = *(unsigned char*)ptr;
      sum += oddbyte;
   }

   sum = (sum >> 16) + (sum & 0xffff);
   sum += (sum >> 16);
   answer = ~sum;

   return answer;
}

struct pseudo_header {
   unsigned int source_address;
   unsigned int dest_address;
   unsigned char placeholder;
   unsigned char protocol;
   unsigned short tcp_length;
};

int main() {
   int sock = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
   if(sock < 0) {
      perror("Error al crear socket");
      return 1;
   }

   char datagram[4096], *data, *pseudogram;
   memset(datagram, 0, 4096);
```

```
    // En Mac usamos struct ip y struct tcphdr con formato BSD
    struct ip *iph = (struct ip *)datagram;
    struct tcphdr *tcph = (struct tcphdr *)(datagram + sizeof(struct ip));

    struct sockaddr_in sin;
    sin.sin_family = AF_INET;
    sin.sin_port = htons(80);

    //
==================================================================
=====
    // ACA MODIFICAS LAS IPs PARA LA CONSIGNA
    //
==================================================================
=====
    // IP Destino (Por ejemplo, un servidor DNS de Google o de la facu)
    sin.sin_addr.s_addr = inet_addr("8.8.8.8");

    // IP Origen inventada (Spoofing) - Distinta a tu localhost
    char source_ip[32] = "192.168.100.200";
    //
==================================================================
=====

    data = datagram + sizeof(struct ip) + sizeof(struct tcphdr);
    strcpy(data, "");

    // Armado del paquete IP (versión Mac/BSD)
    iph->ip_hl = 5;
    iph->ip_v = 4;
    iph->ip_tos = 0;
    iph->ip_len = sizeof(struct ip) + sizeof(struct tcphdr); // Mac requiere host
byte order
    iph->ip_id = htons(54321);
    iph->ip_off = 0;
    iph->ip_ttl = 64;
    iph->ip_p = IPPROTO_TCP;
    iph->ip_src.s_addr = inet_addr(source_ip);
    iph->ip_dst.s_addr = sin.sin_addr.s_addr;
    iph->ip_sum = checksum((unsigned short *)datagram, iph->ip_len);

    // Armado del paquete TCP (versión Mac/BSD)
    tcph->th_sport = htons(1234);
    tcph->th_dport = htons(80);
    tcph->th_seq = htonl(0);
    tcph->th_ack = 0;
    tcph->th_off = 5;
    tcph->th_flags = TH_SYN; // Flag SYN
    tcph->th_win = htons(5840);
```

```
    tcph->th_sum = 0;
    tcph->th_urp = 0;

    struct pseudo_header psh;
    psh.source_address = iph->ip_src.s_addr;
    psh.dest_address = iph->ip_dst.s_addr;
    psh.placeholder = 0;
    psh.protocol = IPPROTO_TCP;
    psh.tcp_length = htons(sizeof(struct tcphdr));

    int psize = sizeof(struct pseudo_header) + sizeof(struct tcphdr);
    pseudogram = malloc(psize);

    memcpy(pseudogram, &psh, sizeof(struct pseudo_header));
    memcpy(pseudogram + sizeof(struct pseudo_header), tcph, sizeof(struct
tcphdr));

    tcph->th_sum = checksum((unsigned short *)pseudogram, psize);

    int one = 1;
    if (setsockopt(sock, IPPROTO_IP, IP_HDRINCL, &one, sizeof(one)) < 0) {
        perror("Error en setsockopt (IP_HDRINCL)");
        return 1;
    }

    if (sendto(sock, datagram, iph->ip_len, 0, (struct sockaddr *)&sin, sizeof(sin))
< 0) {
        perror("Error al enviar el paquete");
    } else {
        printf("Paquete TCP SYN spoofed enviado desde %s hacia %s\n",
source_ip, inet_ntoa(sin.sin_addr));
    }

    free(pseudogram);
    close(sock);
    return 0;
}
EOF
```

El programa desarrollado tiene como objetivo construir y enviar un paquete de red desde cero, saltándose el procedimiento habitual del sistema operativo mediante el uso de raw sockets. Para esto, se arman manualmente dos bloques de información: el encabezado IP, donde se configura una dirección IP de origen falsa y se define la IP de

destino real. Y el encabezado TCP, donde se establecen los puertos y, lo más importante, se activa la bandera SYN (TH_SYN), la cual simula el inicio de una conexión real.

Antes de inyectarlo en la red, el programa calcula los checksums para garantizar que los enrutadores no descarten el paquete.

Para comprobar el funcionamiento, la prueba se ejecutó coordinando dos terminales en simultáneo, una con el programa compilado y otra en la que se dejó corriendo la herramienta tcpdump. La herramienta logró interceptar el paquete y arrojó el siguiente registro: 18:09:27.238111 IP 192.168.100.200.1234 > 8.8.8.8.80: Flags [S], seq 0, win 5840, length 0

Se puede ver que el IP Spoofing se aplicó con éxito. El sistema de monitoreo registró físicamente la salida de un paquete proveniente de la IP inventada (192.168.100.200, puerto 1234) dirigiéndose al servidor de destino (8.8.8.8, puerto HTTP 80). Finalmente, el indicador Flags [S] verifica que el paquete conservó la instrucción de apertura de conexión.